

Análisis y Monitoreo de Sistemas Operativos con Virtualización y Contenedores

Cristian Olarte

Ana Vargas

Andrés Badillo

Abril 2025

Resumen

Este documento describe el desarrollo completo de un entorno híbrido de pruebas para el segundo corte del curso de Sistemas Operativos. El proyecto tiene como objetivo consolidar el uso de herramientas de virtualización, contenedores, monitoreo y gestión de recursos en sistemas Linux. Se utilizan máquinas virtuales QEMU, contenedores Docker y herramientas como Prometheus, Grafana y Zabbix, con interconexiones por subredes personalizadas. El desarrollo de este entorno permite a los estudiantes fortalecer sus habilidades en la administración de sistemas, la gestión de redes y el análisis del rendimiento.

Índice

1. Introducción	2
2. Actividad 1: Instalación de Máquinas Virtuales	2
2.1. Creación de las máquinas virtuales	2
2.2. Configuración de red	5
2.2.1. Verificación de la conexión	5
2.3. Herramientas de Análisis en VMs	7
3. Actividad 2: Instalación de Contenedores Docker	7
3.1. Herramientas en Contenedores	7
4. Actividad 3: Contenedor Central (Fedora)	8
5. Actividad 4: Análisis desde Contenedor Central	8
6. Actividad 5: Arquitectura con Prometheus, Grafana y Zabbix	8
6.1. Integración de Prometheus con Grafana	10
6.1.1. Configuración de Grafana para visualizar métricas de Prometheus	11
7. Conclusiones	12

1. Introducción

Este documento describe el desarrollo completo de un entorno híbrido de pruebas para el segundo corte del curso de Sistemas Operativos. El proyecto tiene como objetivo consolidar el uso de herramientas de virtualización, contenedores, monitoreo y gestión de recursos en sistemas Linux. Se utilizan máquinas virtuales QEMU, contenedores Docker y herramientas como Prometheus, Grafana y Zabbix, con interconexiones por subredes personalizadas. El desarrollo de este entorno permite a los estudiantes fortalecer sus habilidades en la administración de sistemas, la gestión de redes y el análisis del rendimiento.

2. Actividad 1: Instalación de Máquinas Virtuales

Se instalaron tres sistemas operativos utilizando QEMU/KVM y Virt-Manager:

- Rocky Linux
- Manjaro Linux
- Arch Linux

2.1. Creación de las máquinas virtuales

Para la creación de máquinas virtuales, se siguió el siguiente procedimiento:

1. Para comenzar, se hace clic en .“Archivo”

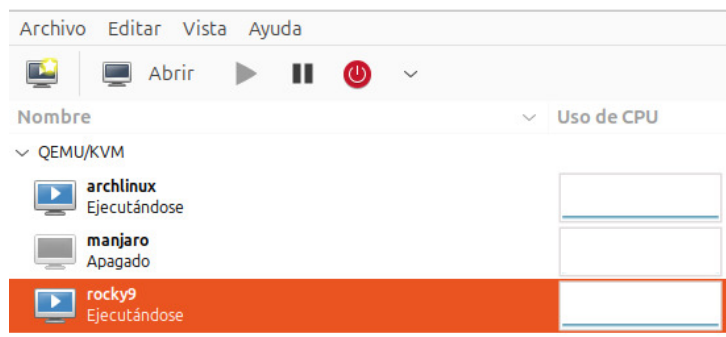


Figura 1: Seleccionar archivo

2. Después de hacer clic en archivo, se selecciona .“Añadir conexión”

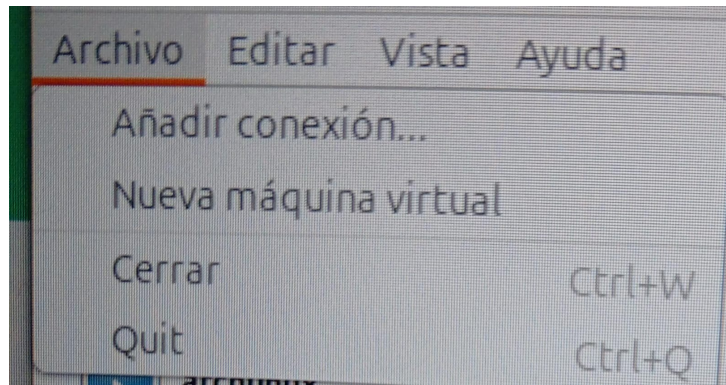


Figura 2: Añadir conexión

3. A continuación, se selecciona la opción "Medio de instalación"

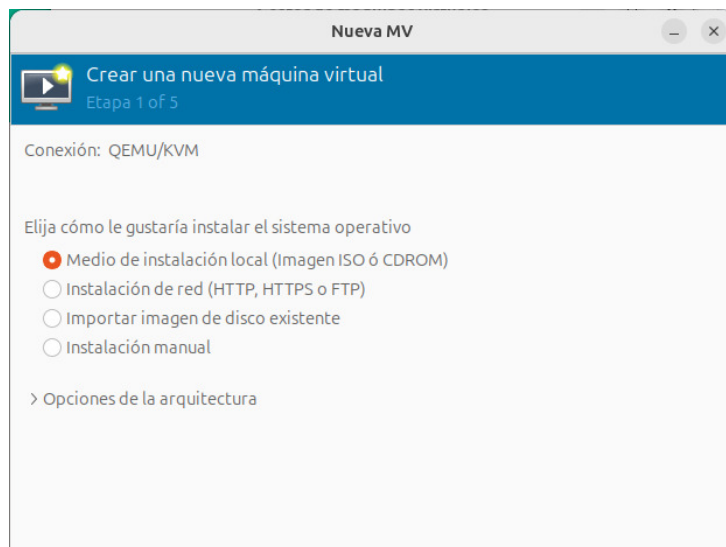


Figura 3: Medio de instalación

4. Se selecciona la imagen ISO y se marca la casilla de detección automática desde el medio de instalación o fuente

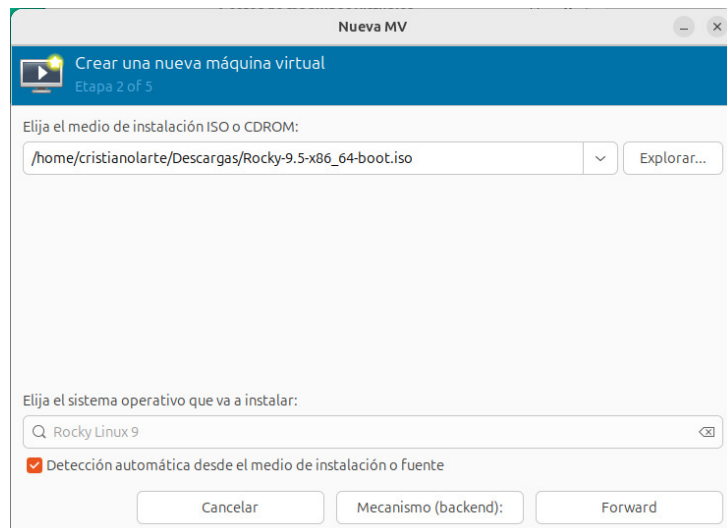


Figura 4: Selección de ISO

5. Se asigna memoria y CPUs (se recomiendan 2 CPUs y 4096MB de memoria)

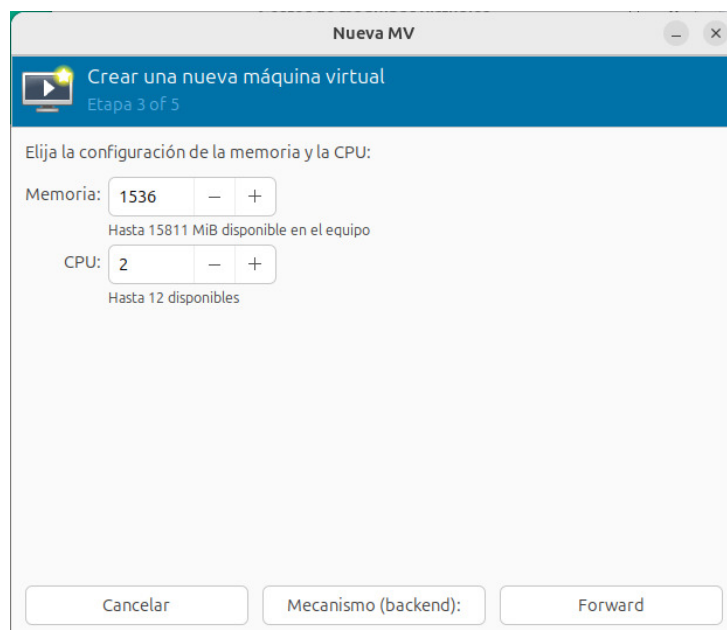


Figura 5: Asignación de recursos

6. Se asigna almacenamiento (se recomienda más de 15 GiB)

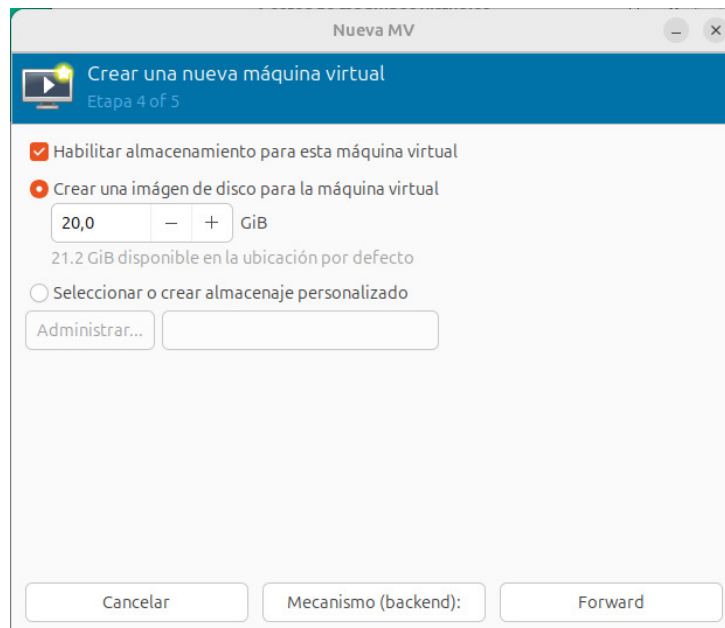


Figura 6: Almacenamiento

7. En la ventana final, se hace clic en "Finalizar" se espera a que se complete la instalación

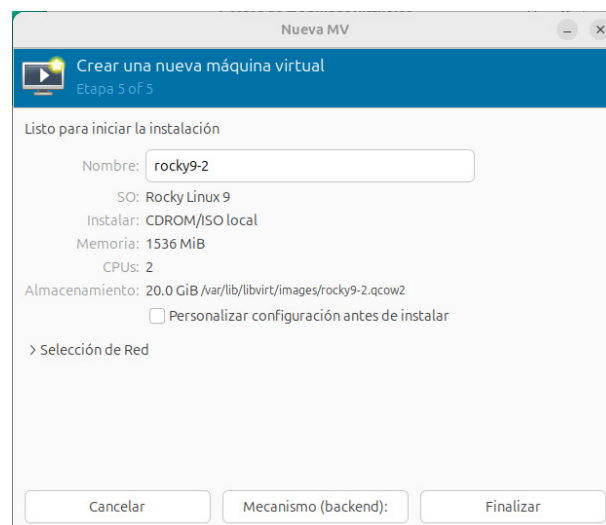


Figura 7: Diagrama de subred para las VMs

2.2. Configuración de red

Subred de VMs: Se asignó la red 192.168.100.0/24 y se creó un puente (br0) para conectarlas con el host y los contenedores.

Crearemos la red br0 y conectaremos las VMs a esa red en puente.

2.2.1. Verificación de la conexión

Hacemos un Nmap para verificar que se conectaron correctamente a la red br0:

```

~$ ping 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=0.382 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=0.401 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=0.356 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=0.401 ms
64 bytes from 192.168.100.1: icmp_seq=5 ttl=64 time=0.385 ms
64 bytes from 192.168.100.1: icmp_seq=6 ttl=64 time=0.386 ms
64 bytes from 192.168.100.1: icmp_seq=7 ttl=64 time=0.279 ms
64 bytes from 192.168.100.1: icmp_seq=8 ttl=64 time=0.398 ms
64 bytes from 192.168.100.1: icmp_seq=9 ttl=64 time=0.410 ms

```

Figura 8: Nmap resultado 1

```

[root@a0595812d17f /]# nmap -sS 192.168.100.11
Starting Nmap 7.92 ( https://nmap.org ) at 2025-04-15 04:03 UTC
Nmap scan report for 192.168.100.11
Host is up (0.000054s latency).
All 1000 scanned ports on 192.168.100.11 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 52:54:00:02:11:04 (QEMU virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.31 seconds

```

Figura 9: Nmap resultado 2

```

[cristianolarte@localhost ~]$ ping 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data.
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=0.193 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=0.333 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=0.302 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=0.387 ms
64 bytes from 192.168.100.1: icmp_seq=5 ttl=64 time=0.425 ms
64 bytes from 192.168.100.1: icmp_seq=6 ttl=64 time=0.413 ms
64 bytes from 192.168.100.1: icmp_seq=7 ttl=64 time=0.378 ms
64 bytes from 192.168.100.1: icmp_seq=8 ttl=64 time=0.428 ms
64 bytes from 192.168.100.1: icmp_seq=9 ttl=64 time=0.437 ms
64 bytes from 192.168.100.1: icmp_seq=10 ttl=64 time=0.437 ms
64 bytes from 192.168.100.1: icmp_seq=11 ttl=64 time=0.190 ms
64 bytes from 192.168.100.1: icmp_seq=12 ttl=64 time=0.342 ms
64 bytes from 192.168.100.1: icmp_seq=13 ttl=64 time=0.346 ms
64 bytes from 192.168.100.1: icmp_seq=14 ttl=64 time=0.416 ms
64 bytes from 192.168.100.1: icmp_seq=15 ttl=64 time=0.411 ms
64 bytes from 192.168.100.1: icmp_seq=16 ttl=64 time=0.423 ms

```

Figura 10: Nmap resultado 3

```

[root@a0595812d17f /]# nmap -sS 192.168.100.12
Starting Nmap 7.92 ( https://nmap.org ) at 2025-04-15 04:09 UTC
Nmap scan report for 192.168.100.12
Host is up (0.00066s latency).
Not shown: 988 filtered tcp ports (no-response), 10 filtered tcp p
PORT      STATE SERVICE
22/tcp    open  ssh
9090/tcp   closed zeus-admin
MAC Address: 52:54:00:80:3D:D1 (QEMU virtual NIC)

```

Figura 11: Nmap resultado 4

```
[cristianolarte@archlinux ~]$ ping 192.168.100.1
PING 192.168.100.1 (192.168.100.1) 56(84) bytes of data:
64 bytes from 192.168.100.1: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 192.168.100.1: icmp_seq=2 ttl=64 time=0.345 ms
64 bytes from 192.168.100.1: icmp_seq=3 ttl=64 time=0.337 ms
64 bytes from 192.168.100.1: icmp_seq=4 ttl=64 time=0.339 ms
^C
```

Figura 12: Nmap resultado 5

```
[root@a0595812d17f /]# nmap -sS 192.168.100.13
Starting Nmap 7.92 ( https://nmap.org ) at 2025-04-15 04:12 UTC
Nmap scan report for 192.168.100.13
Host is up (0.000018s latency).
All 1000 scanned ports on 192.168.100.13 are in ignored states.
Not shown: 1000 closed tcp ports (reset)
MAC Address: 52:54:00:FD:A9:49 (QEMU virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds
```

Figura 13: Nmap resultado 6

2.3. Herramientas de Análisis en VMs

- **Monitoreo de sistema:** glances, bpytop, journalctl
- **Análisis de red:** wireshark, iftop, nethogs
- **Gestión de espacio:** ncd, baobab, tree, rsync

3. Actividad 2: Instalación de Contenedores Docker

Se desplegaron tres contenedores Linux:

- Garuda Linux
- Alpine Linux
- Debian

Subred de Contenedores: Se creó la red Docker red_br0 tipo macvlan, con IPs fijas por contenedor.

3.1. Herramientas en Contenedores

- **Hardware y sistema:** lshw, inxi, lsblk, udevadm
- **Monitoreo:** Netdata
- **Análisis de logs:** lnav, goaccess
- **Gestión de Docker:** ctop, lazydocker

4. Actividad 3: Contenedor Central (Fedora)

Se creó una imagen Docker con Fedora como sistema base, con el siguiente Dockerfile:

```
1 FROM fedora:latest
2 RUN dnf -y install htop net-tools iputils python3 git
3 CMD ["/bin/bash"]
```

Listing 1: Dockerfile para contenedor central con Fedora

Comandos para construir y ejecutar:

```
1 docker build -t central-fedora .
2 docker run -dit --name central --network red_br0 --ip 192.168.100.50
   central-fedora
```

Listing 2: Comandos de construcción y ejecución

5. Actividad 4: Análisis desde Contenedor Central

Desde el contenedor Fedora Central se realizaron:

- **Escaneo de red:** nmap
- **Conectividad:** ping, traceroute
- **Servicios activos:** netstat -tulpn
- **Interfaces IP:** ip a, docker inspect
- **Uso de recursos:** htop
- **Captura de tráfico:** tcpdump

6. Actividad 5: Arquitectura con Prometheus, Grafana y Zabbix

Se desplegaron los servicios de monitoreo con Docker Compose:

```
1 version: '3.8'
2 networks:
3   red_br0:
4     external: true
5 services:
6   prometheus:
7     image: prom/prometheus
8     ports:
9       - "9090:9090"
10    networks:
11      red_br0:
12        ipv4_address: 192.168.100.60
13    volumes:
14      - ./prometheus.yml:/etc/prometheus/prometheus.yml
15
16 grafana:
17   image: grafana/grafana
```

```
18   ports:
19     - "3000:3000"
20   networks:
21     red_br0:
22       ipv4_address: 192.168.100.30
23   volumes:
24     - grafana-data:/var/lib/grafana
25
26   zabbix-server:
27     image: zabbix/zabbix-server-pgsql:latest
28     networks:
29       red_br0:
30         ipv4_address: 192.168.100.62
31     volumes:
32       - grafana-data:
```

Listing 3: docker-compose.yml para servicios de monitoreo

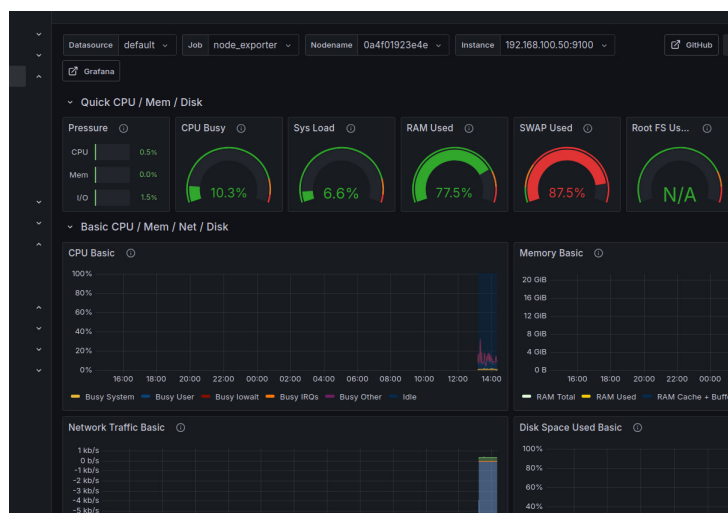


Figura 14: Dashboard de monitoreo

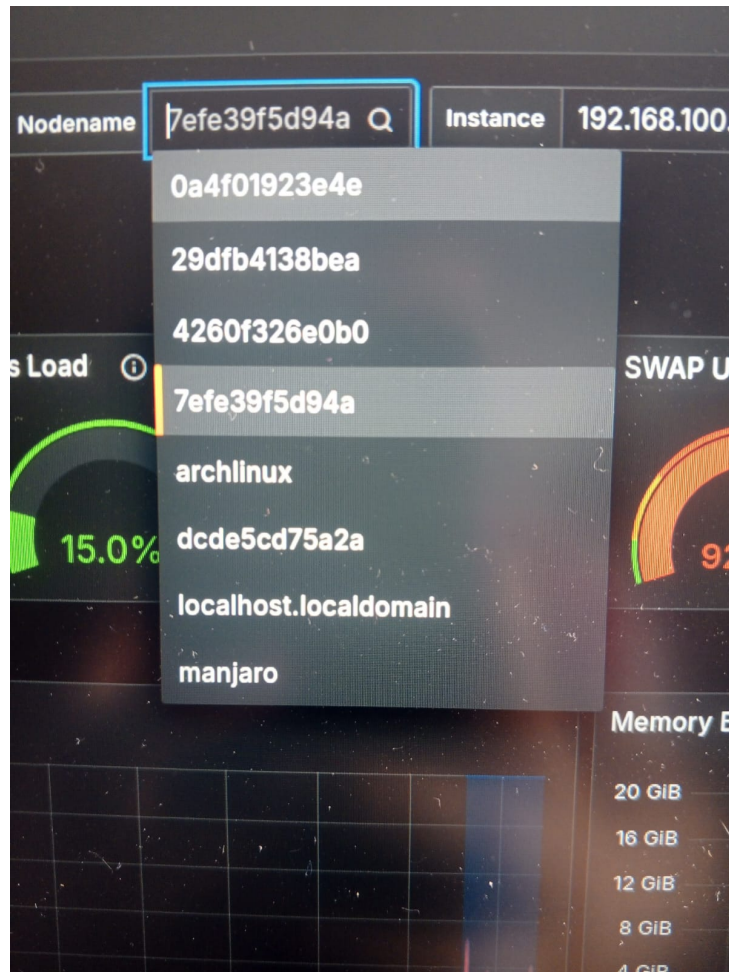


Figura 15: Métricas de rendimiento

6.1. Integración de Prometheus con Grafana

Prometheus se configuró como fuente de datos para Grafana, permitiendo visualizar el rendimiento de las máquinas virtuales en tiempo real. La integración se realizó mediante los siguientes pasos:

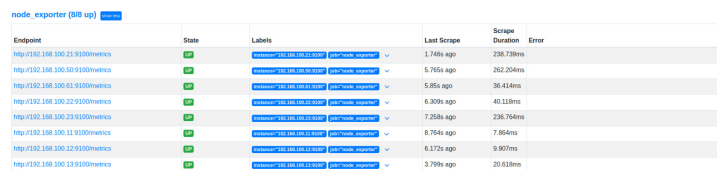
1. Se configuró Prometheus para recopilar métricas de las VMs (Rocky Linux, Manjaro y Arch Linux) a través de node-exporter instalado en cada máquina.
2. En Grafana, se añadió Prometheus como fuente de datos:

```
1 # URL de conexión en la configuración de Grafana
2 http://192.168.100.60:9090
3
```

Listing 4: Configuración de la fuente de datos en Grafana

3. Se crearon dashboards personalizados en Grafana para visualizar:
 - Uso de CPU y memoria de cada VM
 - Tráfico de red entre contenedores y VMs
 - Espacio en disco y operaciones I/O

- Tiempo de actividad y carga del sistema



Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.100.21:9100/metrics	UP	instance="192.168.100.21" job="node_exporter"	1.748s ago	238.728ms	
http://192.168.100.50:9100/metrics	UP	instance="192.168.100.50" job="node_exporter"	5.765s ago	262.204ms	
http://192.168.100.61:9100/metrics	UP	instance="192.168.100.61" job="node_exporter"	5.85s ago	36.424ms	
http://192.168.100.22:9100/metrics	UP	instance="192.168.100.22" job="node_exporter"	6.309s ago	40.118ms	
http://192.168.100.23:9100/metrics	UP	instance="192.168.100.23" job="node_exporter"	7.258s ago	236.764ms	
http://192.168.100.11:9100/metrics	UP	instance="192.168.100.11" job="node_exporter"	8.764s ago	7.864ms	
http://192.168.100.12:9100/metrics	UP	instance="192.168.100.12" job="node_exporter"	6.172s ago	9.907ms	
http://192.168.100.13:9100/metrics	UP	instance="192.168.100.13" job="node_exporter"	3.799s ago	20.638ms	

Figura 16: Dashboard de monitoreo

6.1.1. Configuración de Grafana para visualizar métricas de Prometheus

Para visualizar las métricas de rendimiento de las máquinas virtuales en Grafana, se siguió el siguiente procedimiento de configuración:

1. **Acceso a Grafana:** Se ingresó a la interfaz web de Grafana mediante la URL `http://192.168.100.30:3000` utilizando las credenciales por defecto (usuario: admin, contraseña: admin).
2. **Configuración de la fuente de datos:**
 - En el menú lateral, se navegó a **Configuration > Data Sources**
 - Se hizo clic en **Add data source**
 - Se seleccionó **Prometheus** de la lista de fuentes disponibles
 - En el campo **URL**, se ingresó `http://192.168.100.60:9090`
 - Se dejó el resto de configuraciones con sus valores predeterminados
 - Se hizo clic en **Save & Test** para verificar la conexión exitosa
3. **Creación de un dashboard:**
 - Se navegó a **Create > Dashboard**
 - Se seleccionó **Add new panel**
 - En el editor de consultas, se seleccionó **Prometheus** como fuente de datos
 - Para monitorear el uso de CPU, se ingresó la consulta PromQL:

```

1      100 - (avg by (instance) (irate(node_cpu_seconds_total{
2      mode="idle"}[1m])) * 100)

```

Listing 5: Consulta PromQL para uso de CPU

- Para la memoria RAM, se utilizó:

```

1      100 * (1 - ((node_memory_MemFree_bytes +
2      node_memory_Cached_bytes +
3      node_memory_Buffers_bytes) / node_memory_MemTotal_bytes))

```

Listing 6: Consulta PromQL para uso de memoria

- Se personalizó el panel con título, unidades (porcentaje) y rango de tiempo
- Se guardó el panel con el botón **Apply**

4. Agregación de paneles adicionales:

- Se repitió el proceso para crear paneles de monitoreo de:
 - Carga del sistema (`node_load1`, `node_load5`, `node_load15`)
 - Uso de disco (`node_filesystem_avail_bytes / node_filesystem_size_bytes`)
 - Tráfico de red (`irate(node_network_receive_bytes_total[1m])`, `irate(node_network_receive_bytes_total[1m])`)
- Se organizaron los paneles en un diseño coherente mediante arrastrar y soltar

5. Configuración de variables de plantilla:

- Se agregó una variable `instance` para filtrar por máquina virtual
- En la configuración de la variable:

```
1   label_values(node_uname_info, instance)
2
```

Listing 7: Configuración de variable de instancia

- Esto permitió cambiar entre las diferentes máquinas virtuales (Rocky Linux, Manjaro y Arch Linux) desde un menú desplegable

6. Guardado del dashboard:

- Se guardó el dashboard completo con el nombre "Monitoreo VMs"
- Se configuró el intervalo de actualización automática a 10 segundos

Una vez completada la configuración, el dashboard mostraba en tiempo real los datos de rendimiento de las tres máquinas virtuales, permitiendo identificar patrones de consumo de recursos, fluctuaciones en la carga del sistema y posibles cuellos de botella en el rendimiento. Esta integración proporciona una visión unificada del rendimiento de todo el entorno virtualizado, facilitando la identificación de cuellos de botella y la optimización de recursos del sistema.

7. Conclusiones

Se logró implementar un entorno completo de monitoreo distribuido usando contenedores Docker y herramientas de virtualización. La integración de Prometheus, Grafana y Zabbix permitió analizar el rendimiento y la disponibilidad de sistemas de forma centralizada.